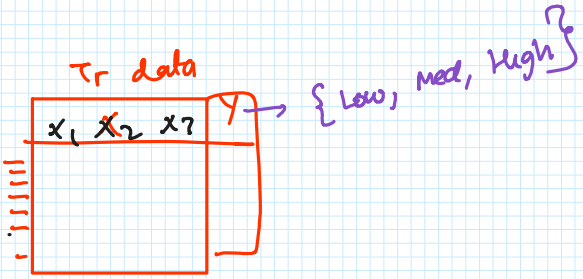
KNN - 2

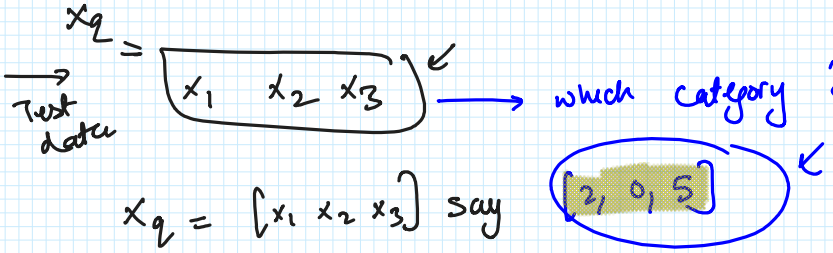
08 October 2025 21:01

Agenda →
- Quick Recap
- K → Hyperparameter Tuning
- Time complexity
- Categorical features
- Distance Metrics
- KNN imputation

Quick Revise

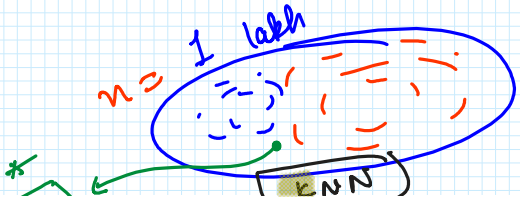
K

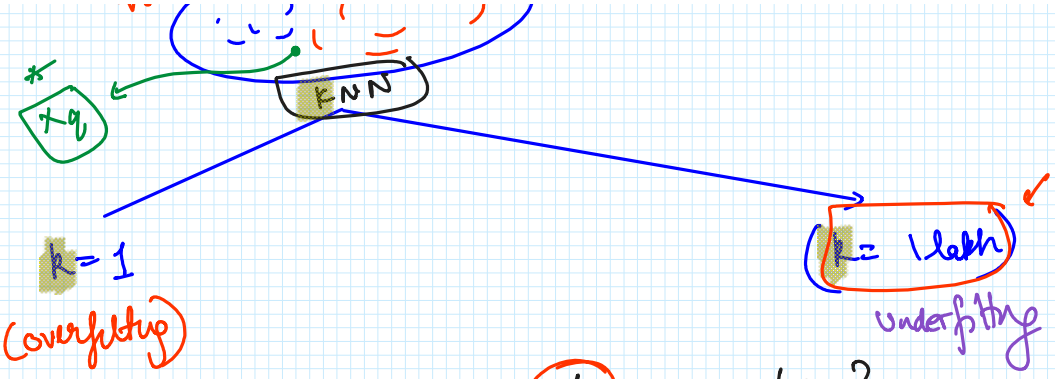




Steps
1. Dist (x_q) and all other rows [Euclidean]
2. Sort distance & select top "k" rows
3. Select the most probable label

Bias – Variance Trade off { How changing k affects model perf → Bias & Variance changes)

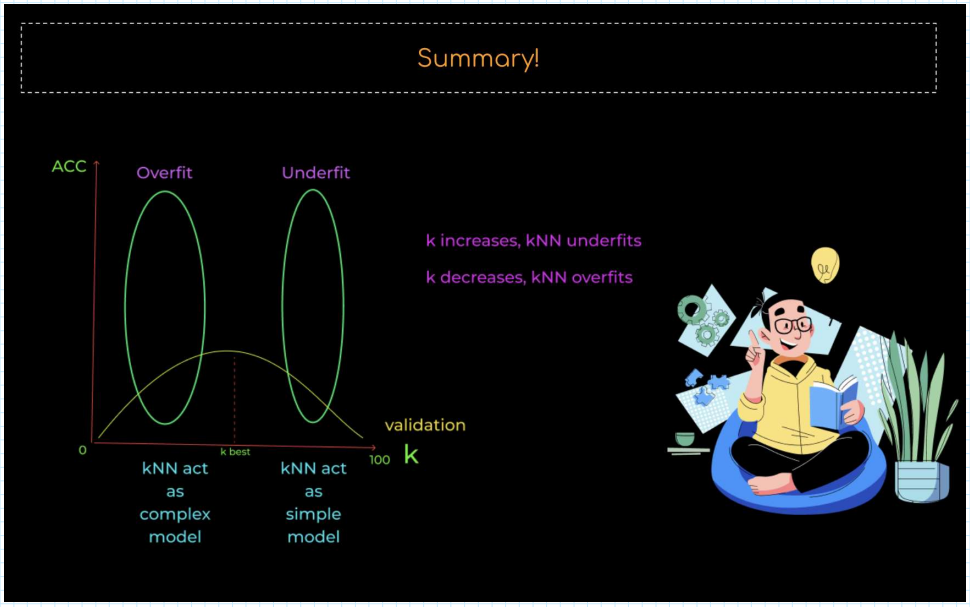


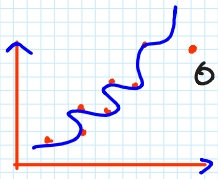


(overfitting)

Underfitting

Q Which case corresponds to U/F and O/F ?





O/F fitted Model → Bias Problem or Variance

well on Tr data

Not on unseen data

Under fitted model → Bias Problem → Error in tr data

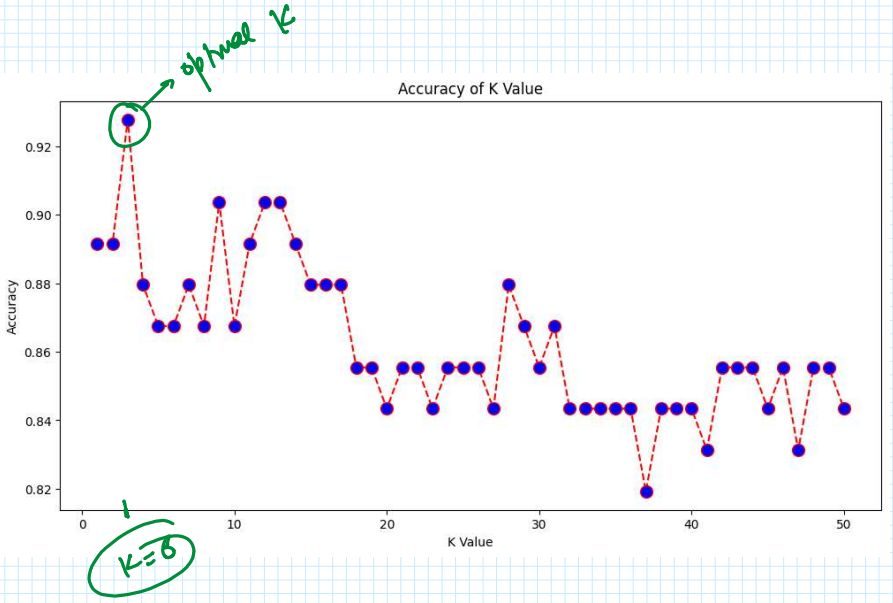
Lets do hyperparameter Tuning (K)

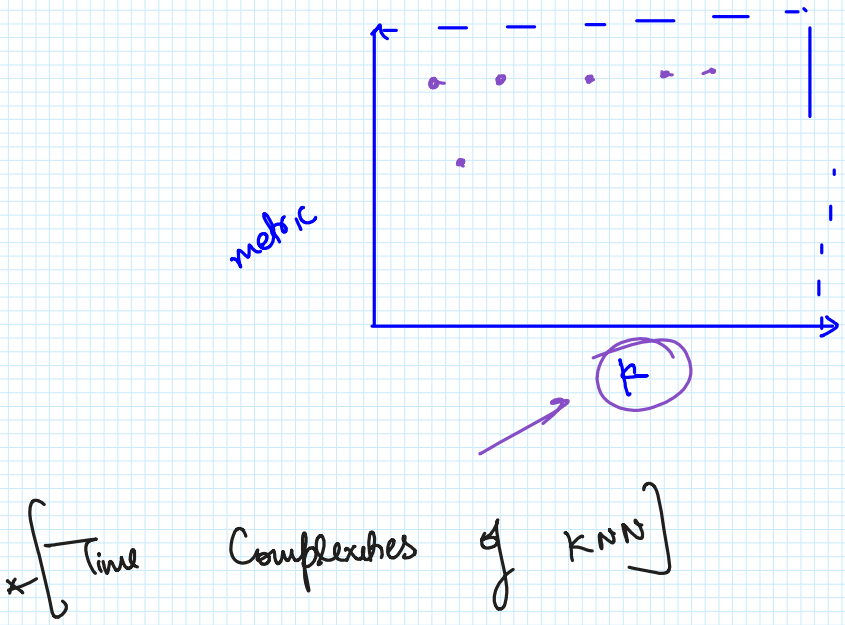
Small k →

- Small `k` (e.g., 1–3):
  - Model fits the training data tightly.
  - Low bias, high variance (overfitting risk).
- Large `k` (e.g., 20–50):
  - Model averages over more neighbors.

- Low bias, high variance (overfitting risk).
- **Large `k` (e.g., 20–50):**
  - Model averages over more neighbors.
  - High bias, low variance (underfitting risk).

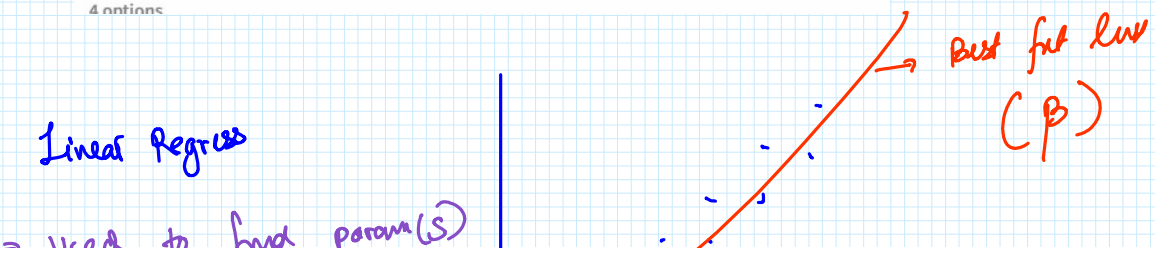
Large K





quiz(what do you think) What will be the training time complexity of kNN ?

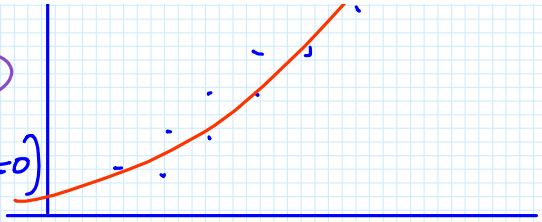
4 options



→ used to find param(s)

$[w_1 x_1 + w_2 x_2 + \dots + w_d x_d + w_0 = 0]$

$\vec{w} = [w_1 \; - \; - \; w_d \; w_0]$

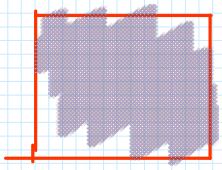


KNN

KNN is a **lazy learning algorithm** — it doesn't learn an explicit model; it memorizes the training data and makes predictions based on the "nearness" of points.

* [There is no training]

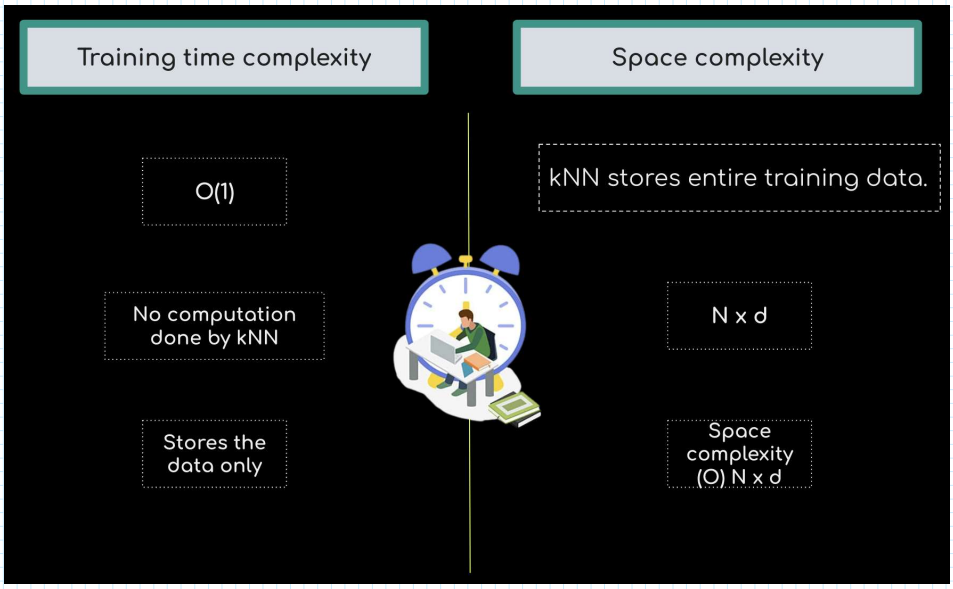
Predict(testpt)

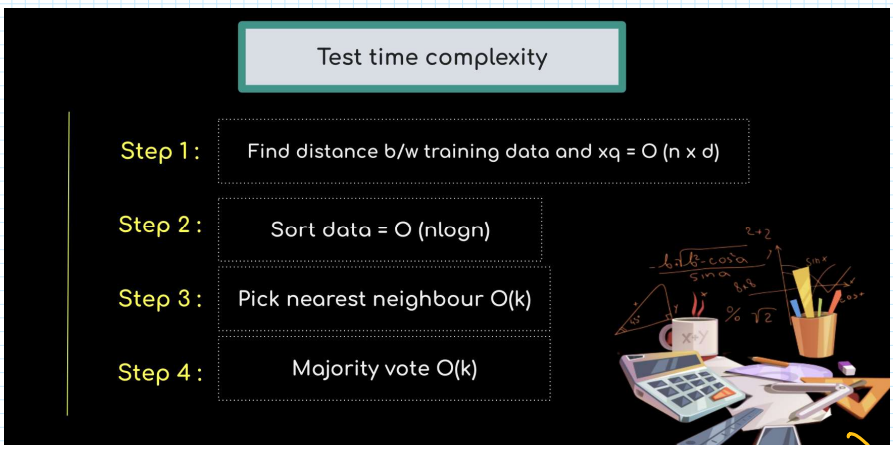


→ space comp (query train)

$O(n * d)$







Step 4 : Majority vote O(k)



As k << n & d, hence O(k) ignored
Time complexity = O (nd + nlogn)

O(nd + nlogn)

Time Complexity at II testing time (x_q)

① use x_q and all X $\quad X = \begin{bmatrix} & & \\ & & \\ & & \\ & & \end{bmatrix}$

$O(n*d)$

x_q $(1, 2, 0)$

② Sort distances $O(n \log n)$

③ Pick k NN $O(K)$

KNN → for categorical features

↳ (distance based)

A, B, O, AB
+ -

X

Gender	Age	BP	Glucose level	Blood group	Diabetic or Not
M/F	Num	Num	Num	8 categories	Y
+2 columns				+8	

\# Soln → (encode these categories)

ML → OHE

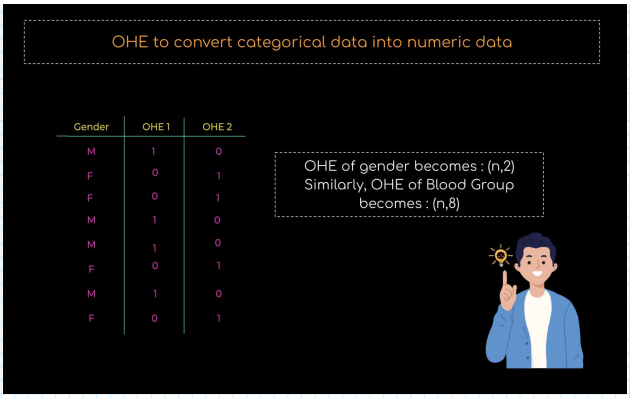
OHE to convert categorical data into numeric data

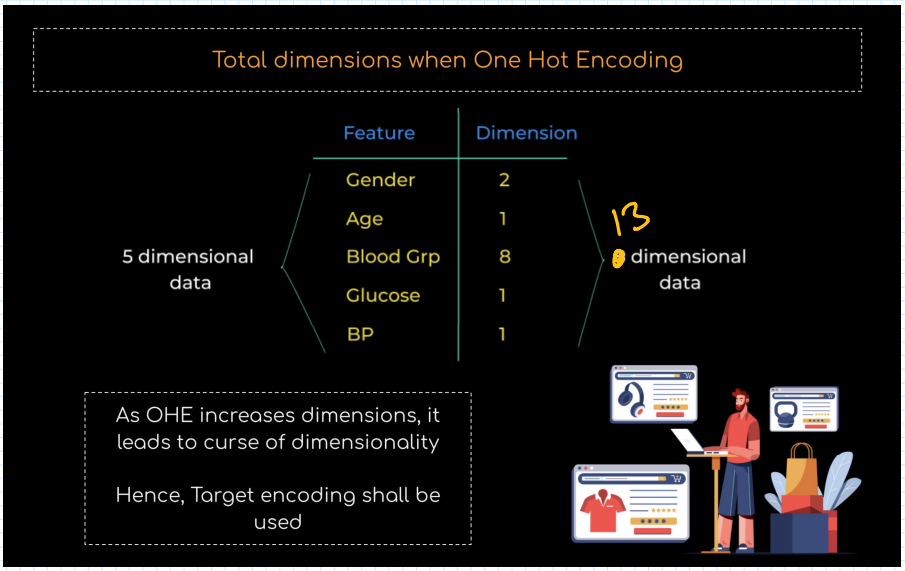
Gender	OHE 1	OHE 2
M	1	0
F	0	1
F	0	1
M	1	0
M	1	0
F	0	1
M	1	0
F	0	1

OHE of gender becomes : (n,2)
Similarly, OHE of Blood Group becomes : (n,8)



ML → OHE





quiz (what do you think) if One Hot Encoding increases data dimension to (d=1000), will Eculidean Distance work ?

Theoretically → Yes

Realistically → NO

Euclidean ≤ $\sqrt{d}$

Curse of Dimensionality

→ (Distance looses its meaning in high dimensions)

Use Some Better distance metrics

Use Some Better distance metrics

1. Euclidean
2. Manhattan Dist
3. Cosine Similarity

10:11 pm

What other distance to use?

Minkowski Distance

Euclidean $= [\sum_{j=1}^{d}(x_j^{(1)} - x_j^{(2)})^2]^{\frac{1}{2}} \rightarrow x^2$ same as $|x|^2$

- similar to L2 Norm $\sum_{j=1}^{d} w_j^2$, hence called L2 distance

Manhattan $= [\sum_{j=1}^{d}(x_j^{(1)} - x_j^{(2)})^1]^1$

- similar to L1 Norm $\sum_{j=1}^{d} |w_j|$, hence called L1 distance

Generalized Equation $= [\sum_{j=1}^{d}(x_j^{(1)} - x_j^{(2)})^p]^p$ if $p = 1 \rightarrow$ Manhattan, if $p = 2 \rightarrow$ Euclidean

10:11 pm

Ex

	x_1	x_2	x_3	Y
P1	2	3	5	A
P2	0	0	-2	B

x_1 x_2 x_3

$x_q = 0, 1, 2$

Cosine Sim $= 1 - \frac{a \cdot b}{|a||b|}$

$(x_q - P_1)$

ED $x_q - P_1$

$$\left[(2-0)^2 + (3-1)^2 + (5-2)^2\right]^{1/2}$$

Manhattan Dist

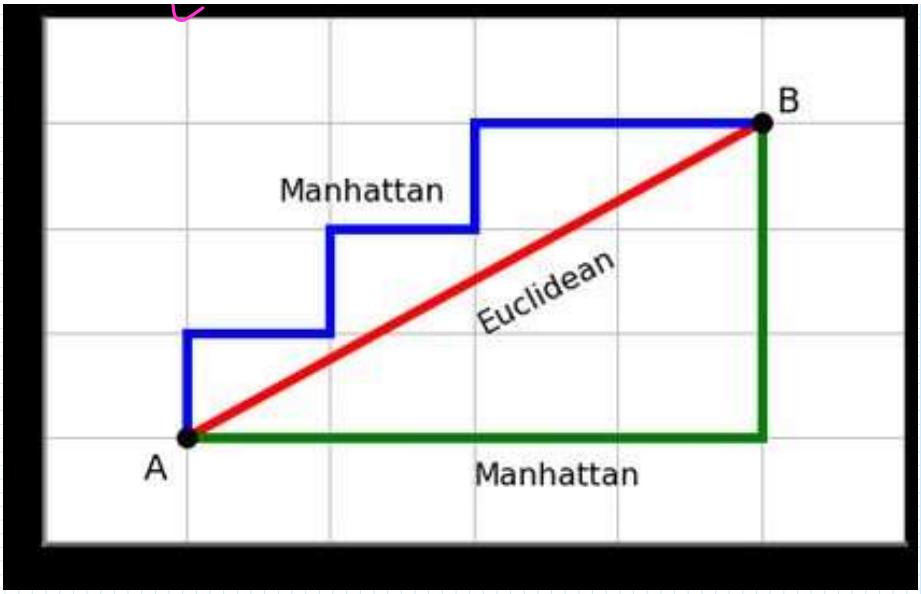
$$\left[|2-0|^1 + |3-1|^1 + |5-2|^1\right]^{1/1} = (2+2+3) = 7$$

Minkowski (p)

$$\left[(2-0)^p + (3-1)^p + (5-2)^p\right]^{1/p}$$

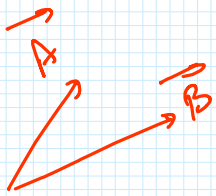
$p = 2$ (Euclidean)

$p = 1$ (Manhattan)



Cosine Distance

Cosine distance (or cosine dissimilarity) is a **measure of how different two vectors are** based on the **angle between them**, not their magnitude.



$\cos\theta$ (similarity) $= \frac{\vec{A} \cdot \vec{B}}{|A|\,|B|}$ = cosine Similarity

Cosine Distance = 1 − cosine Similarity

2, 3, 5

0, 1, 2

$\cos\theta = \frac{2\times0 + 3\times1 + 5\times2}{\sqrt{2^2+3^2+5^2}\ \sqrt{0^2+1^2+2^2}}$

Cos Dis = 1 − $\cos\theta$

$1 - 0.8 = 0.2$

Euc Newto $\cos\theta$ = high = 0.8 Similarity

$\vec{a}\cdot\vec{b} = |a||b|\cos\theta$

$\cos\theta = \frac{\vec{a}\cdot\vec{b}}{|a||b|}$

Distance metric used for kNN

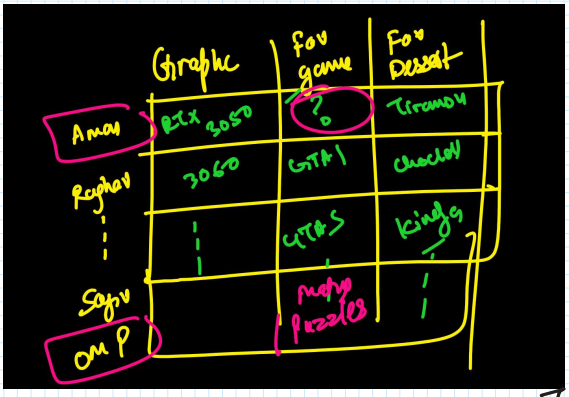
- Euclidean Distance – for low dimensional data
- Cosine similarity — for high dimensional data
- Manhattan – useful when data is like a map
- Minkowski – for using custom distance metric

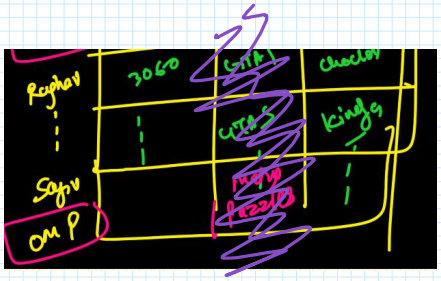
Uber



*[KNN → for missing data]







Top 5 most similar

P1 P7 P8 P20 P50

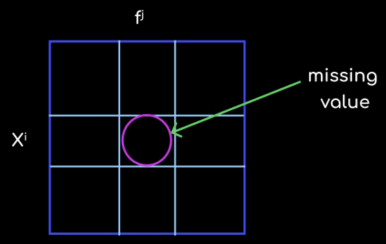
Aman → Fav Game

Avg (most similar game)

What are the techniques of imputing?

- Mean or median of Fj feature
- Analyzing data and manually impute value
- Mean and median of whole data

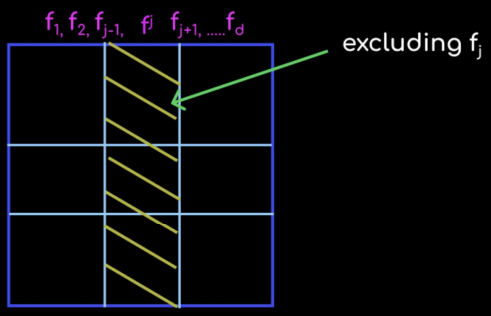




kNN for Imputation

Step 1 : Exclude fj from data





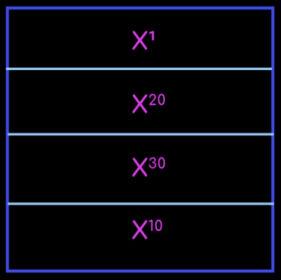
kNN for Imputation

Step 2: Find distance between xi and rest data — k nearest neighbour

Suppose x^1, x^{20} ,x^{10}, x^{30} were nearest neighbours

if k=4





kNN for Imputation

Step 3: For these nearest neighbour, check value for fj value

$$X^{i}_{fj} = Avg\ (\ X^{1}_{fj}\ ,\ X^{20}_{fj},\ X^{30}_{fj},\ X^{10}_{fj}\ )$$



optional →

How to speed up search at Production

Local Sensitive Hashing (LSH)

Google images use kNN to provide famous monuments just by searching city.

kNN work so fast in Google searches?

By hashing algorithm —LSH (Locality Sensitive Hashing)



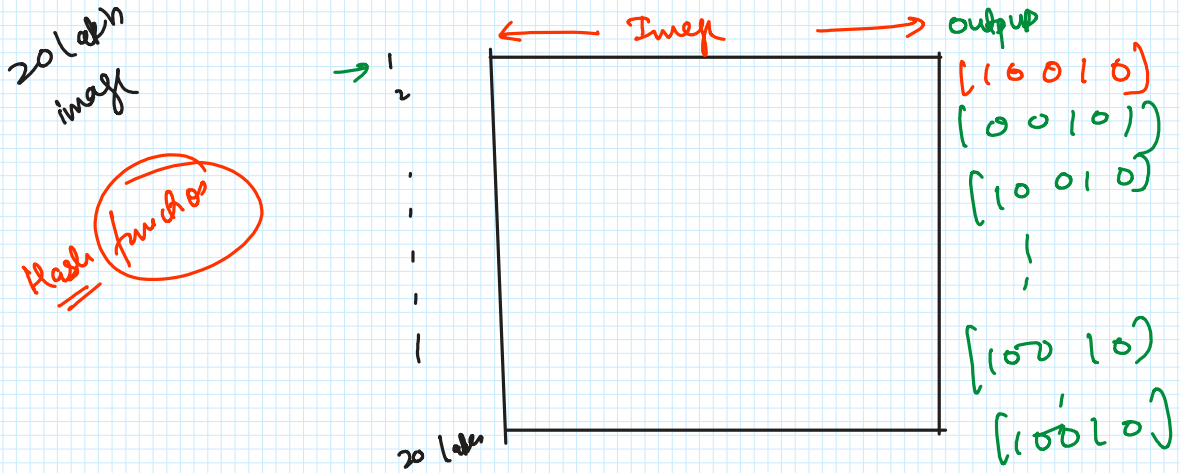
(Hash) Hash fn

How does LSH work?

For hash table create randomised hash function (h(x))

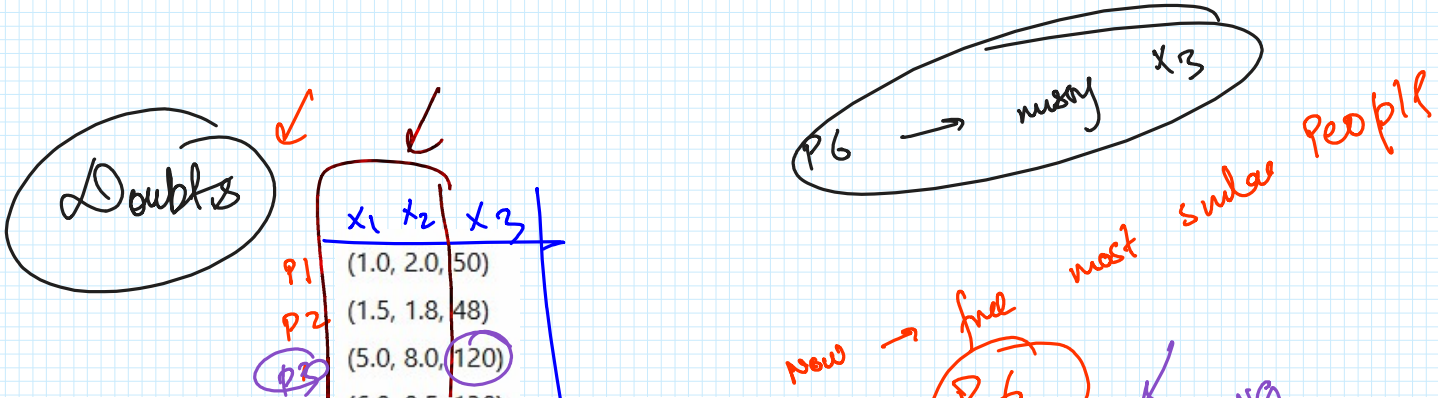
	f^1	f^2	f^3
X^1	5	10	7
X^2	-1	0	-10
X^3	-20	20	20
X^4	100	20	30

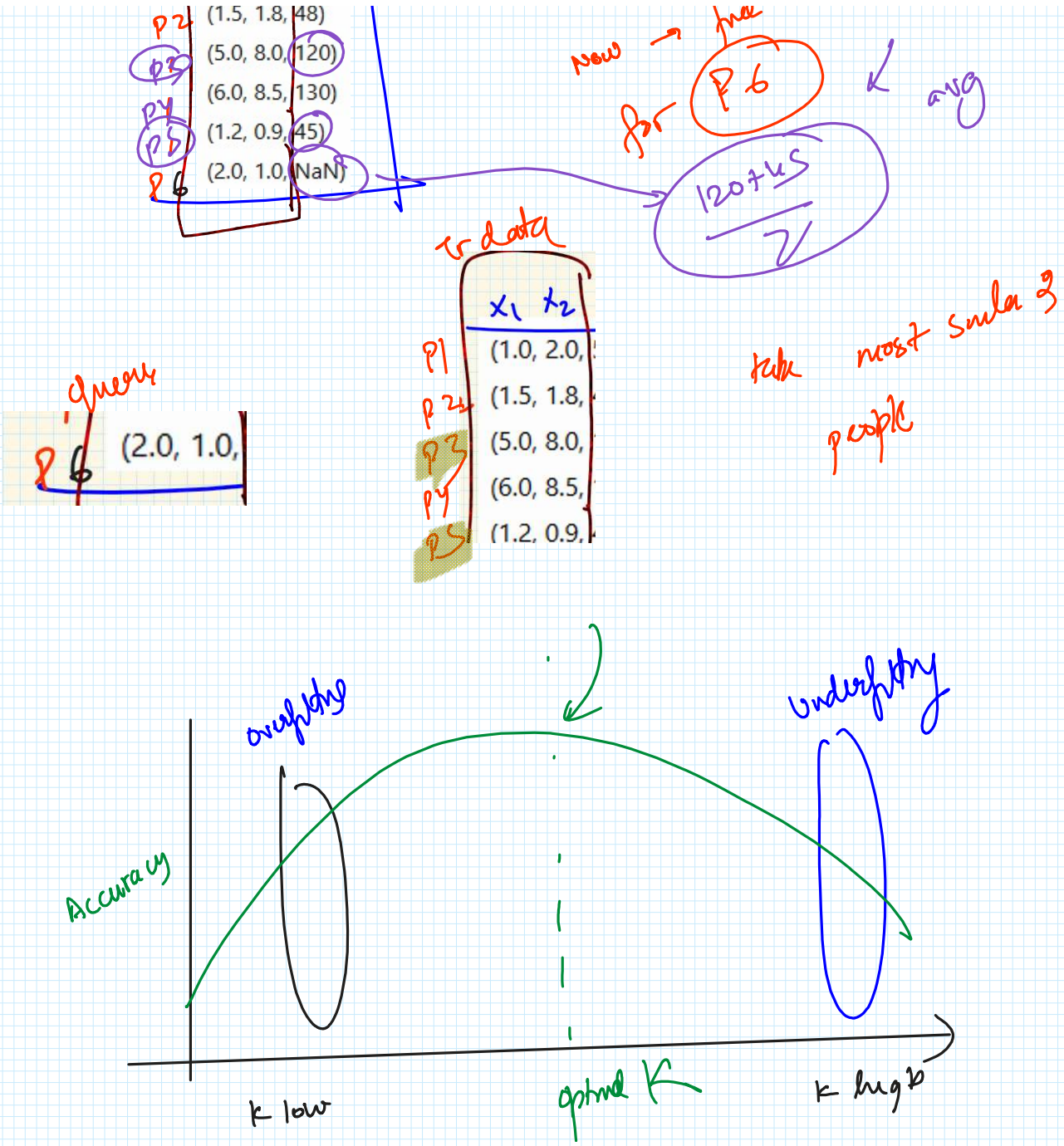
Gives key for hash table

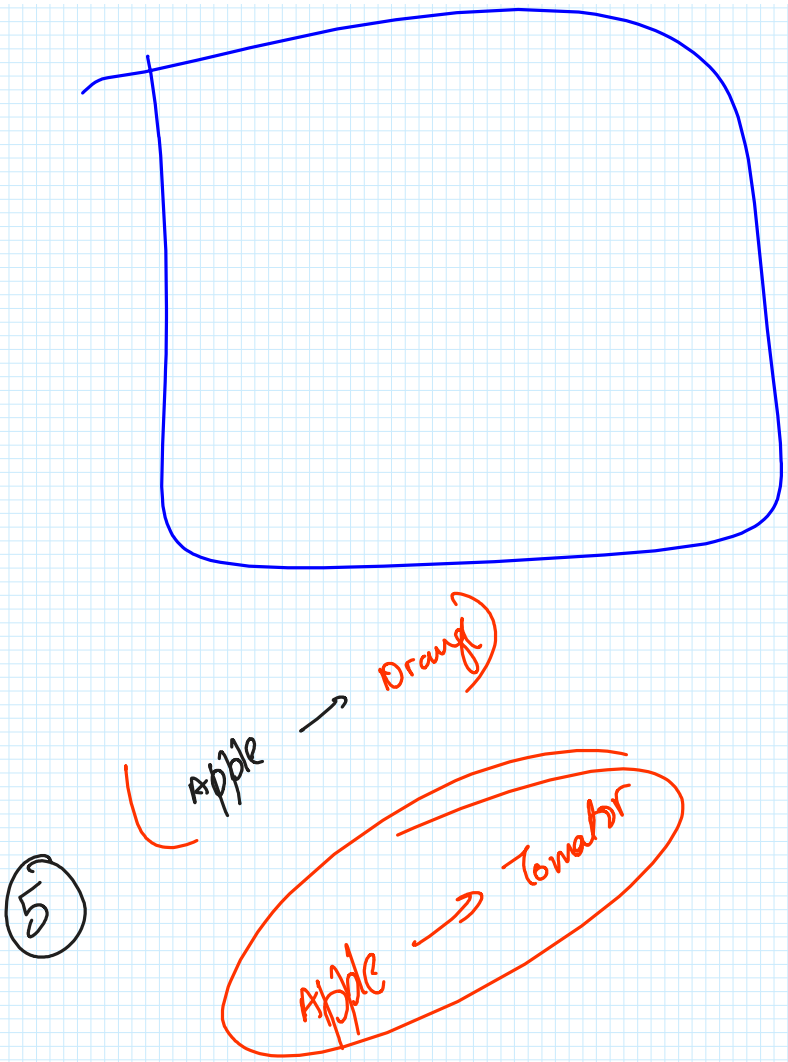


query "Delhi moments" $h(x)$ → [1 00 1 0]

Hash → Tries to cluster / group similar things




(1.5, 1.8, 48)
(5.0, 8.0, 120)
(6.0, 8.5, 130)
(1.2, 0.9, 45)
(2.0, 1.0, NaN)
now
for
P6
avg
120+45
2
tr data
x1 x2
(1.0, 2.0,
(1.5, 1.8,
(5.0, 8.0,
(6.0, 8.5,
(1.2, 0.9,
query
(2.0, 1.0,
take most similar 3 people
overfitting
underfitting
Accuracy
K low
optimal K
K high


Apple → Orange
5
Apple → Tomato